Perl: данные, переменные и функции

```
my $language = "Perl";
print "$language\n";
```

Новые языки программирования

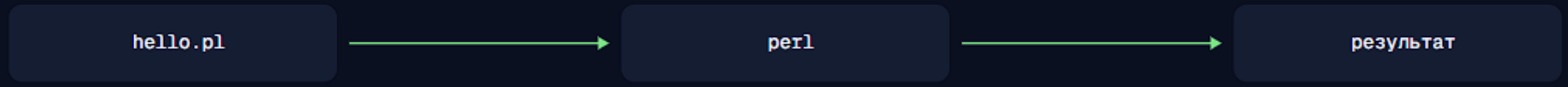
Как запустить Perl-программу

```
use strict;
use warnings;

print "Hello, Perl!\n";
```

```
perl hello.pl
perl -v
```

- Perl — интерпретируемый язык
- исходный файл обычно имеет расширение .pl
- запуск выполняется через интерпретатор perl
- текущая стабильная ветка: Perl 5.44.x



От системных скриптов к универсальному языку

1987
появился Perl

1994
Perl 5

2026
Perl 5.44.x

```
while (<>) {
    print if /error/i;
}
```

- автор: Larry Wall
- обработка текста и создание отчётов
- системное администрирование и разработка ПО
- Perl 5 — основа современной ветки

Сильная сторона Perl — работа с текстом и регулярными выражениями

Что делает Perl необычным?

```
my $text = "Error: file not found";

if ($text =~ /error/i) {
    print "Найдена ошибка\n";
}
```

- регулярные выражения встроены в язык
- scalar/list context меняет поведение кода
- процедурный, функциональный и ОО-стиль

Актуальная стабильная ветка: Perl 5.44.x

динамическая типизация

регулярные выражения

контекст вычисления

There's More Than One Way…

Три основных формы данных

SCALAR

одно значение

ARRAY

список значений

HASH

пары ключ → значение

```
my $name = "Alice";
my @numbers = (10, 20, 30);
my %user = (
    name => "Alice",
    age  => 21
);
```

- scalar начинается с $
- array — с @
- hash — с %
- тип связан с формой доступа к данным

Ссылки позволяют строить вложенные структуры данных

Строки и числа: разные операции

```
my $a = "10";
my $b = 5;

print $a + $b;      # 15
print $a . $b;      # 105
```

- + — числовая операция
- . — конкатенация строк
- == сравнивает числа
- eq сравнивает строки

Важно понимать контекст значения

Типы часто преобразуются автоматически

```
my $text = "42";

my $number = $text + 8;
my $string = "$number";

print $number;   # 50
print $string;   # "50"
```

- неявное преобразование
- строка может использоваться как число
- число может использоваться как строка
- преобразование можно сделать явно



my, our и область видимости

```
my $x = 10;

{
    my $x = 20;
    print $x;      # 20
}

print $x;          # 10
```

- my создаёт лексическую переменную
- переменная видна в своей области
- внутренний блок скрывает внешнюю
- our — доступ к package-переменным

Внешняя область

$x = 10



Внутренний блок

$x = 20

В Perl нет модели владения как в Rust

```
my $a = [1, 2];
my $b = $a;

push @$b, 3;

print "@$a";    # 1 2 3
```

- нет строгой модели ownership/borrowing
- $b — ссылка на тот же массив
- изменение видно через обе ссылки
- память: подсчёт ссылок



Функции: входные и выходные данные

```
sub add {
    my ($a, $b) = @_;
    return $a + $b;
}

my $result = add(2, 3);
print $result;     # 5
```

- функция объявляется через sub
- аргументы доступны через @_
- return возвращает результат
- можно вернуть несколько значений

```
return (10, 20);
my ($a, $b) = get_data();
```



Рекурсивные функции

```
sub factorial {
    my ($n) = @_;

    return 1 if $n <= 1;

    return $n * factorial($n - 1);
}

print factorial(5);   # 120
```

- функция вызывает саму себя
- нужно условие остановки
- механизм знаком по другим языкам



Замыкание сохраняет переменную из внешней области

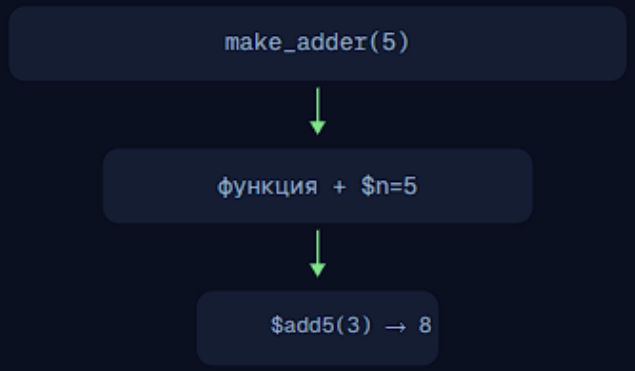
```
sub make_adder {
    my $n = shift;

    return sub {
        return $_[0] + $n;
    };
}

my $add5 = make_adder(5);

print $add5->(3);    # 8
```

- функция возвращает другую функцию
- внутренняя функция использует $n
- $n живёт после make_adder
- это и есть closure



ИТОГ

Что главное в Perl?

ТЕКСТ

Perl исторически ориентирован на обработку текста

ДАННЫЕ

$ / @ / % + ссылки

КОНТЕКСТ

Операции зависят от способа использования значения

ФУНКЦИИ

@_ → return → рекурсия → замыкания

Perl сочетает динамическую типизацию, мощную работу с текстом и необычную модель данных